

低成本 CCI/DDI 网络 + Laplacian + 稀疏图路径 Unbalanced SR-OT 方案 v4

修正说明：恢复原报告模板；网络命名统一为 `light_cci`；补充 LR 规模估算与 `profile-only`；统一 KNN、辅助图与最终 OT 成本符号；详细解释 sparse unbalanced OT。

根据当前代码中 `clean_expression_cci_mix`、`laplacian`、`sr_ot`、`_ot_common` 的结构整理

1 本版先给结论

本版建议把后续实验收敛为一条低成本、可解释、不会跨层泄露的主线：

light_cci 网络 + Laplacian/HKS 节点特征 + 稀疏图路径候选 + Unbalanced SR-OT

它的目标是同时解决三个问题：

1. **特征构造太重**：不再构造 `expression + CCI` 大特征矩阵，先用 CCI/DDI 图本身生成轻量结构特征。
2. **OT 成本太粗**：不只用简单加权平均距离，而是用辅助图上的最短路径代价，让 PIJ 更像沿 CCI/DDI 网络传播。
3. **PIJ 二次复杂度太重**：不默认构造完整 dense 成本矩阵，而是先用候选筛选把全连接配对降到稀疏 support，再在 support 上求 OT。

本版不再把 JointNMF 作为主线讨论，也不再引入 Fused Gromov-Wasserstein 这类高成本图匹配方法。主结果中，各层级独立构造自己的 PIJ，不把 lower-to-upper 映射、domain label 或 overlap mapping 放进 lower 层 PIJ，避免 EI 比较时发生跨层信息泄露。

2 LR 文件数量不够：必须看文件大小与 nnz

只知道有多少个 LR 文件是不够的。当前 COMMOT 输出里，每个 ligand-receptor pair 通常对应一个稀疏矩阵文件。不同 LR 文件的大小可以差很多：有些 LR pair 只产生少量非零通讯，有些 LR pair 会产生大量 sender-receiver 非零项。因此，运行时间更应该由 LR 稀疏矩阵的非零元规模估计，而不是单纯由 LR 文件数估计。

2.1 LR 文件里面是什么

对某个 LR pair 记为 r ，COMMOT 导出的文件可以理解为一个非负稀疏矩阵：

$$M^{(r)} \in \mathbb{R}_+^{n \times n}, \quad (1)$$

$$M_{uv}^{(r)} > 0. \quad (2)$$

这里, r 表示某个 ligand-receptor pair; n 表示当前样本、当前层级的 unit 数量; u 表示 sender unit; v 表示 receiver unit; $M_{uv}^{(r)}$ 表示 sender u 通过 LR pair r 对 receiver v 的通讯强度。

配套的 manifest 文件一般类似 *_COMMOT_lr_pairs.tsv。里面通常会出现 filename、nnz、shape_0、shape_1、ligand、receptor、pathway 等字段。这里 filename 指向对应的 LR 稀疏矩阵文件; nnz 是该矩阵的非零元数量; shape_0 与 shape_1 是矩阵维度。

2.2 LR 文件规模如何估算运行时间

light_cci 建图时, 代码不是只数 LR 文件名, 而是会读 manifest, 再读取每个 LR 矩阵。一个合理的时间估算可以写成:

$$T_{\text{CCI_build}} \approx c_{\text{io}} \sum_r \text{file_size}_r + c_{\text{nnz}} \sum_r \text{nnz}_r + c_{\text{edge}} \hat{E}_{\text{CCI}}. \quad (3)$$

这里, $T_{\text{CCI_build}}$ 表示 CCI/DDI 建图耗时估计; c_{io} 表示读取文件的单位代价; file_size_r 表示第 r 个 LR 文件大小; c_{nnz} 表示扫描非零项的单位代价; nnz_r 表示第 r 个 LR 稀疏矩阵的非零元数量; c_{edge} 表示生成图边的单位代价; $\hat{E}_{\text{CCI}}$ 表示预计最终展开出的通讯图边数量。

注意, $\hat{E}_{\text{CCI}}$ 不一定等于 $\sum_r \text{nnz}_r$ 。如果 ligand 或 receptor 是复合基因名, 一个非零通讯项可能会展开成多个 ligand gene 与 receptor gene 的组合, 然后再被 shared genes、表达阈值、target expression、cci_min 等规则过滤。因此可以再粗略估计为:

$$\hat{E}_{\text{CCI}} \approx \sum_r \text{nnz}_r \cdot e_r \cdot \rho_r. \quad (4)$$

这里, $e_r = |L_r| |R_r|$ 表示第 r 个 LR pair 的基因展开倍数; L_r 是 ligand 侧基因集合; R_r 是 receptor 侧基因集合; ρ_r 表示表达过滤和阈值过滤后的保留比例。

2.3 必须新增 profile-only 脚本

正式构造网络和 PIJ 之前, 应新增一个只做规模审计的脚本, 例如:

```
scripts/profile_light_cci_ot.py
```

这个脚本不求完整特征、不跑 Sinkhorn, 只输出输入规模与风险估计。建议输出字段如下:

类别	建议字段	用途
CCI/LR 规模	n_lr_files, total_lr_nnz, mean/median/max_lr_nnz, total_lr_bytes, max_lr_file	估算 LR 读取、CCI 扫描和 CCI 建图时间。
节点和图边	n_units, estimated_cci_edges, graph_edges, avg_degree, max_degree	判断 Laplacian 是否会慢、图是否过密。
dense OT 风险	$n_s, n_t, n_s n_t$, dense_cost_bytes	判断完整成本矩阵是否会爆内存。
sparse OT 规模	K , K_{rev} , estimated_candidates, sparse_cost_bytes	判断稀疏候选 OT 是否可跑。
时间估计	scan_lr_seconds_est, build_graph_seconds_est, laplacian_seconds_est, ot_seconds_est	正式跑之前给粗略时间范围。
安全检查	是否读取跨层 mapping; 是否把 mapping 用进 PIJ; candidate coverage	防止层级泄露和候选过窄。

3 网络命名与边界: light_cci

新网络建议直接命名为 light_cci。这个名字比 clean_expression_cci_mix 更准确，因为新主线不再构造 expression + CCI 大矩阵特征，也不再混入旧 GRN。它只保留轻量通讯图结构。

```
network_method = "light_cci"
```

light_cci 的原则如下:

- **节点:** 当前层级的 unit。spot/cell 层就是 spot 或 cell; domain 层就是 domain。
- **边:** 当前层级内的通讯边。cell/spot 层用 CCI; domain 层用 DDI 或 domain 聚合后的通讯边。
- **特征:** 不输出大 expression block 作为主特征; Laplacian/HKS 直接从图邻接矩阵得到节点结构特征。
- **旧 GRN:** 这条主线先不用旧 GRN 混合。只有当研究问题明确切到基因层时, 再单独讨论跨细胞基因调控图。
- **泄露边界:** 不同层级各自独立构造 PIJ。主结果中, 不把 lower-to-upper 映射、domain label、overlap mapping 放进 lower 层 PIJ。

4 Laplacian 特征与 KNN 候选不是重复压缩

Laplacian/HKS 的作用是把每个节点在图里的结构位置变成低维向量。KNN 候选的作用不是再压缩一次，而是在 source-target 全连接配对中删掉明显不合理的候选对。两者解决的是不同问题。

模块	输入	输出	解决的问题
Laplacian/HKS	单个时间点、单个层级的 <code>light_cci</code> 图邻接矩阵	每个节点一个低维结构特征 z_i	降低节点表征维度，避免巨大 expression/CCI 特征。
KNN 候选	source 特征 z_i 、target 特征 z'_j 、可选 SR/空间差异	候选集合 Ω	降低 OT 配对数量，避免 $n_s \times n_t$ 全连接。
Sparse OT	候选集合 Ω 和成本 C_{ij}	稀疏 PIJ 矩阵 P	只在可信候选边上求 transport coupling。

因此，KNN 不会改变原图，也不会把 Laplacian 特征再降维。它只是规定：source 节点 i 不再和所有 target 节点 j 尝试匹配，而只保留少量最可能的 j 。真正的风险不是“重复压缩”，而是 K 太小导致真对应被删掉。这个风险可以用 reverse KNN、coverage 报告和小样本 dense 对照来控制。

4.1 KNN 的“近邻”怎么取

先定义一个只用于候选筛选的预成本 B_{ij} ：

$$B_{ij} = \alpha_z d_z(z_i, z'_j) + \alpha_s d_s(q_i, q'_j) + \alpha_x d_x(x_i, x'_j). \quad (5)$$

这里， i 表示 source 时间 t_0 的节点； j 表示 target 时间 t_1 的节点； z_i 和 z'_j 分别表示 source 与 target 节点的 Laplacian/HKS 结构特征； q_i 和 q'_j 表示 SR 或发育状态标量； x_i 和 x'_j 表示空间坐标； d_z 、 d_s 、 d_x 分别表示归一化后的结构距离、SR 距离和空间距离； α_z 、 α_s 、 α_x 是非负权重。

然后对每个 source 节点选 top- K target，对每个 target 节点反向选 top- K_{rev} source，二者取并集：

$$\Omega_{\text{forward}} = \{(i, j) : j \in \text{topK}_j(B_{ij})\}, \quad (6)$$

$$\Omega_{\text{reverse}} = \{(i, j) : i \in \text{topK}_i(B_{ij})\}, \quad (7)$$

$$\Omega = \Omega_{\text{forward}} \cup \Omega_{\text{reverse}}. \quad (8)$$

这里， Ω_{forward} 保证每个 source 至少有候选 target； Ω_{reverse} 防止某些 target 完全没有候选流入； Ω 是最终进入辅助图的跨时间候选边集合。

B_{ij} 不是最终 OT 成本。它只用于选候选和给跨时间候选边设置初始边权。最终进入 OT 目标函数的是后面在辅助图上计算的路径成本 C_{ij} 。

5 辅助图 H 的符号统一

上一版里最容易误解的地方是把节点集和边集写在一起取并集。正确写法必须明确区分节点集合与边集合。

$$S = \text{source 时间 } t_0 \text{ 的节点集合,} \quad (9)$$

$$T = \text{target 时间 } t_1 \text{ 的节点集合,} \quad (10)$$

$$E_0 = \text{source 时间内部的 CCI/DDI 图边集合,} \quad (11)$$

$$E_1 = \text{target 时间内部的 CCI/DDI 图边集合,} \quad (12)$$

$$\Omega = \text{source 到 target 的候选跨时间边集合.} \quad (13)$$

辅助图应定义为：

$$H = (V_H, E_H, w_H), \quad (14)$$

$$V_H = S \cup T, \quad (15)$$

$$E_H = E_0 \cup E_1 \cup \Omega. \quad (16)$$

这里， V_H 是辅助图的节点集合； E_H 是辅助图的边集合； w_H 是辅助图上的边权函数。因此是“节点和节点取并集，边和边取并集”，不是把节点集与边集混在一起。

5.1 时间辅助图和 KNN 的关系

KNN 只决定 Ω ，也就是哪些跨时间边可以进入辅助图。辅助图 H 则把三类边合起来：source 内部边 E_0 、target 内部边 E_1 、跨时间候选边 Ω 。之后在 H 上求 source 节点 i 到 target 节点 j 的最短路径距离，把这个路径距离作为最终 OT 成本。

对象	含义	是否最终成本
B_{ij}	预成本，只用于找 KNN 候选，并给跨时间候选边一个初始权重。	不是最终成本。
Ω	由 forward/reverse KNN 得到的跨时间候选边集合。	不是成本,只是 support。
H	把 source 内部图、target 内部图、跨时间候选边合成的辅助图。	不是矩阵,是算路径的图。
C_{ij}	在 H 上算出的 source i 到 target j 的归一化最短路径代价。	是最终 OT 成本。

5.2 取消重复建模

如果先定义跨时间候选边代价 $c_{ij}^\times = \alpha_z d_z + \alpha_s d_s + \alpha_x d_x$ ，最后又定义 $C_{ij} = \lambda_H d_H + \lambda_z d_z + \lambda_s d_s + \lambda_x d_x$ ，就会把同一批信息重复加入两次。新版取消这个重复，只保留一条主链条：

1. 用 B_{ij} 选择候选集合 Ω ;
2. 用 Ω 、 E_0 、 E_1 构造辅助图 H ;
3. 在 H 上计算 $C_{ij} = \text{normalize}(\text{dist}_H(i, j))$;
4. 把 C_{ij} 送入 sparse unbalanced OT。

6 Sparse Geodesic Unbalanced SR-OT：完整方案

这个方案把三个想法合在一起：图最短路径负责“定调”，稀疏候选负责“减少计算量和避免离谱对应”，unbalanced/partial OT 负责“表达发育过程中状态出生、消失和质量不完全守恒”。

6.1 输入和输出

$$G_0 = (S, E_0, w_0), \quad (17)$$

$$G_1 = (T, E_1, w_1), \quad (18)$$

$$z_i, z'_j \text{ 是 source/target 节点的 Laplacian/HKS 特征}, \quad (19)$$

$$q_i, q'_j \text{ 是 source/target 节点的 SR 或发育状态标量}, \quad (20)$$

$$a_i, b_j \text{ 是 source/target 节点质量}, \quad (21)$$

$$P \in \mathbb{R}_+^{|S| \times |T|}. \quad (22)$$

这里， G_0 是 source 时间 t_0 的 `light_cci` 图； G_1 是 target 时间 t_1 的 `light_cci` 图； a_i 表示 source 节点 i 的给定质量； b_j 表示 target 节点 j 的给定质量； P_{ij} 表示从 source 节点 i 到 target 节点 j 的转移强度。

a_i 和 b_j 可以先取均匀质量，也可以取节点大小、spot 数或细胞数。主线建议先用均匀质量或 unit size，避免把过多表达量重新塞回 PIJ。

6.2 Step 1：用 Laplacian/HKS 得到结构特征

对每个时间点、每个层级，先从 `light_cci` 图得到邻接矩阵 A 。再构造归一化 Laplacian：

$$L = I - D^{-1/2} A D^{-1/2}. \quad (23)$$

这里， A 是图邻接矩阵； D 是度矩阵； I 是单位矩阵； L 是 normalized Laplacian。

普通 eigenvector 可以作为快速 sanity check；主结果更建议 HKS，因为 HKS 可以理解为“从每个节点出发，信号在图上扩散一段时间后留下的结构指纹”，它对 eigenvector 的符号翻转更稳。

6.3 Step 2: 用预成本 B_{ij} 构造候选集合

这一步的目标不是求最终距离，而是快速过滤候选。为了省计算，实际实现中可以用 approximate nearest neighbors，而不是显式计算完整 $n_s \times n_t$ 距离矩阵。

$$B_{ij} = \alpha_z d_z(z_i, z'_j) + \alpha_s d_s(q_i, q'_j) + \alpha_x d_x(x_i, x'_j), \quad (24)$$

$$\Omega = \text{topK_by_source}(B) \cup \text{topK_by_target}(B). \quad (25)$$

如果担心 KNN 删错候选，必须输出 coverage 诊断：每个 source 是否至少有 K 个候选；每个 target 是否至少有若干候选流入；候选边覆盖了多少连通分量； $K = 20, 50, 100$ 时 EI 是否稳定。

6.4 Step 3: 构造辅助图并计算图路径成本

辅助图 H 允许一个 source 节点先沿 source 内部通讯图移动，再通过某条跨时间候选边过渡到 target 时间，最后沿 target 内部通讯图移动。这比直接用欧式距离更像“在网络结构上发生发育转移”。

$$H = (V_H, E_H, w_H), \quad (26)$$

$$V_H = S \cup T, \quad (27)$$

$$E_H = E_0 \cup E_1 \cup \Omega. \quad (28)$$

边权函数可以写为：

$$w_H(e) = \begin{cases} \gamma_0 c_0(e), & e \in E_0, \\ \gamma_1 c_1(e), & e \in E_1, \\ B_{ij}, & e = (i, j) \in \Omega. \end{cases} \quad (29)$$

这里， e 是辅助图里的一条边； $c_0(e)$ 是 source 内部边代价； $c_1(e)$ 是 target 内部边代价； γ_0 和 γ_1 控制 source/target 内部移动代价强度； B_{ij} 是跨时间候选边的预成本。

最终成本只保留图路径距离：

$$C_{ij} = \text{normalize}(\text{dist}_H(i, j)). \quad (30)$$

这里， $\text{dist}_H(i, j)$ 表示在辅助图 H 上，从 source 节点 i 到 target 节点 j 的最短路径距离； normalize 表示把不同时间点、不同层级的路径距离缩放到可比范围，例如除以中位数或 95% 分位数。

工程上不需要对所有 i, j 求最短路。可以只为每个 source 保留 H 上最短的 M 个

target, 得到最终稀疏 support Ω_{final} 。这样最终 OT 的候选边数量是 $|S|M$ 量级, 而不是 $|S||T|$ 。

6.5 Step 4: Sparse Unbalanced OT 目标函数

普通 balanced OT 会强制每个 source 的质量全部送出去, 也强制每个 target 的需求全部被满足:

$$P\mathbf{1} = a, \quad (31)$$

$$P^\top \mathbf{1} = b. \quad (32)$$

这里, $P\mathbf{1}$ 是 P 的行和向量, 表示每个 source 实际送出的总质量; $P^\top \mathbf{1}$ 是 P 的列和向量, 表示每个 target 实际收到的总质量; a 是 source 侧给定质量分布; b 是 target 侧给定质量分布。

但是发育过程中, 某些状态可能消失, 某些状态可能新生, 节点数量和类型也可能变化, 所以完全守恒不一定合理。Unbalanced OT 把硬约束改成软惩罚: 如果某个 source 不适合完整延续, 它可以少送一点; 如果某个 target 是新出现的状态, 它也可以不完全来自旧 source。偏离质量守恒不是禁止, 而是要付出惩罚。

$$\min_{P \geq 0, \text{supp}(P) \subseteq \Omega_{\text{final}}} \sum_{(i,j) \in \Omega_{\text{final}}} P_{ij} C_{ij} \quad (33)$$

$$+ \varepsilon \sum_{(i,j) \in \Omega_{\text{final}}} P_{ij} (\log P_{ij} - 1) \quad (34)$$

$$+ \tau_s \text{KL}(P\mathbf{1} \| a) + \tau_t \text{KL}(P^\top \mathbf{1} \| b). \quad (35)$$

逐项解释如下:

- P_{ij} : 最终要学习的 PIJ 元素, 表示从 source 节点 i 到 target 节点 j 的转移质量。
- $\text{supp}(P) \subseteq \Omega_{\text{final}}$: 只允许候选集合里的边有非零 P_{ij} , 其他位置固定为 0。这就是 sparse OT 的关键。
- $\sum P_{ij} C_{ij}$: 运输成本。 P_{ij} 越大, 就越希望 C_{ij} 越小, 也就是尽量把质量送到图路径上更合理的 target。
- $\varepsilon \sum P_{ij} (\log P_{ij} - 1)$: 熵正则项。 ε 越大, P 越平滑; ε 越小, P 越接近硬匹配, 但数值更难。
- $\text{KL}(P\mathbf{1} \| a)$: 如果某个 source 实际送出的质量和它原本质量 a_i 差太多, 就惩罚。
- $\text{KL}(P^\top \mathbf{1} \| b)$: 如果某个 target 实际收到的质量和目标质量 b_j 差太多, 就惩罚。
- τ_s 与 τ_t : 质量守恒强度。越大越接近 balanced OT; 越小越允许状态消失或新生。

直观理解：第一项让转移沿便宜的图路径走；熵项让解稳定；两个 KL 项允许“不是所有 source 都必须完整延续，也不是所有 target 都必须由旧状态完全解释”。这比强制一一对应更符合发育过程。

7 工程实现顺序

建议按下面顺序推进，先保守验证，再逐步加复杂度：

1. **补 profile-only**：先输出 LR/nnz/边数/dense OT 内存/sparse OT 候选规模，避免盲跑。
2. **实现 light_cci 网络**：保留 CCI/DDI 图结构，不构造 expression + CCI 大特征矩阵，不混旧 GRN。
3. **复用现有 Laplacian**：先用普通 Laplacian embedding 跑 sanity check，再加 HKS 稳定版本。
4. **实现 sparse candidate**：先用 B_{ij} 做 forward/reverse KNN，输出 coverage 诊断。
5. **实现 geodesic cost**：构造辅助图 H ，在候选范围内计算 C_{ij} 。
6. **实现 sparse unbalanced OT**：只在 Ω_{final} 上求 PIJ，输出 sparse .npz 与 units.csv。
7. **做小样本 dense 对照**：选几个小时时间对比较 dense SR-OT 与 sparse geodesic unbalanced SR-OT，确认 EI 趋势没有被候选筛选破坏。

主结果必须避免跨层泄露：lower 层 PIJ 不能读取 upper/domain 映射作为成本或候选边约束。映射文件最多用于结果汇总、coverage 审计或单独辅助实验，不能混入主 EI 计算链条。